

Forge

The AI-native developer platform for Cardano

IndiaCodeX Cardano Hackathon Submission

The Problem

- Cardano has excellent individual tools — Aiken, Lucid Evolution, Mesh, Blockfrost — but no project owns the integration between them.
- Every team hand-assembles its own project structure before writing business logic
- Hand-writes TypeScript types with no compiler-enforced link back to the validator
- Hand-rolls its own deployment-tracking scheme, or skips it entirely
- Has no systematic check for eUTxO-specific bugs like double satisfaction

The Solution

```
forge build "Build an escrow smart contract with milestone-based payments"
```

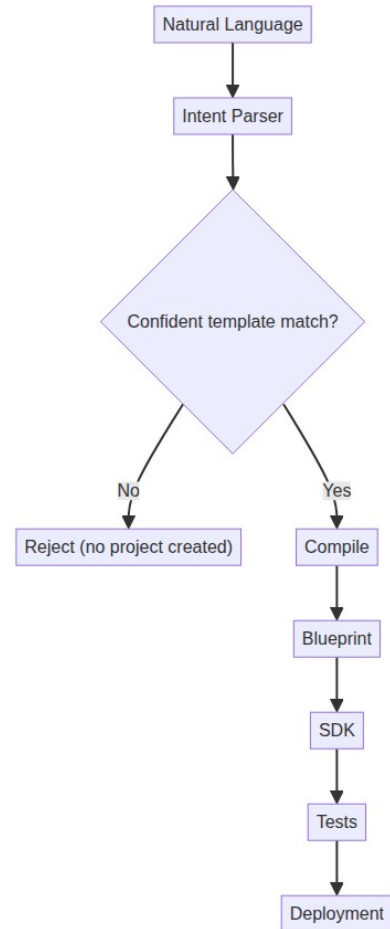
- One command takes a natural-language request and produces:
- A real, compiled Aiken project — verified against the actual compiler
- A real CIP-57 blueprint
- A typed TypeScript SDK, generated from that blueprint
- A passing test against an in-memory emulator
- A real CIP-19 deployment address and versioned manifest
- All in about ten seconds.

Why This Is Trustworthy, Not Just Impressive

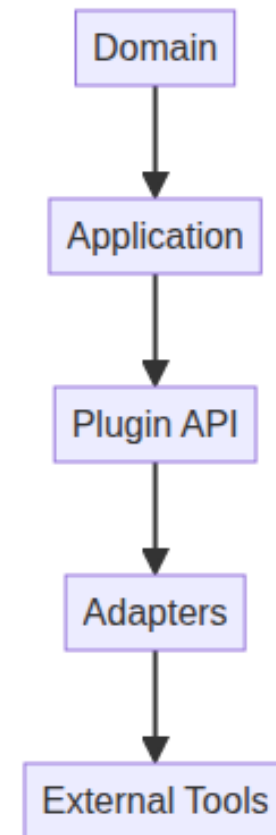
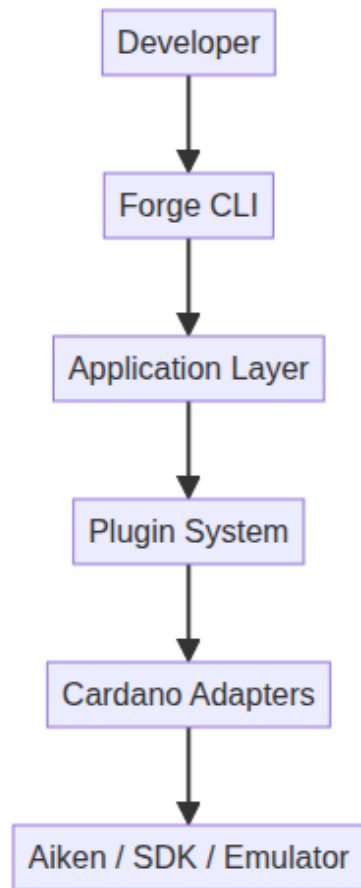
The language model NEVER writes blockchain logic.

- It has exactly two responsibilities anywhere in the platform:
 - 1. Classify intent — which template category matches the description
 - 2. Narrate decisions the system already made deterministically
- A separate, deterministic template engine renders every line of Aiken source.
- A low-confidence request is REJECTED outright — never guessed at.

How Forge Works



Architecture Overview



Supported Templates

1. Escrow with Milestone Payments (spend validator)

Use cases: Freelancing · Construction · Project funding

2. NFT Minting with Royalties (mint validator)

Use cases: NFT marketplaces · Creator royalties

3. Token Vesting (spend validator)

Use cases: Employee token vesting · Investor lockups

Live Demo — Real, Captured Output

```
forge build — escrow-with-milestone-payments

$ forge build "Build an escrow smart contract with milestone-based payments"

Forge — building "Build an escrow smart contract with milestone-based payments"
(The model only classifies intent and extracts parameters — it never writes Aiken source; the deterministic
template engine does.)

→ Scaffolded escrow-smart-contract
→ Compiling with the real Aiken compiler...
  compiled 1 validator(s) — CIP-57 blueprint parsed
→ Generated the typed TypeScript SDK
→ Running tests against the in-memory emulator...
  1 passed, 0 failed
→ Computing the deployment address on preview...
  address: addr_test1wqhkd0vf60ud73jnyysf3j3p3y5m43ey4w6ayp526w226yxsh5zj26

Done.

Project:      /tmp/forge-demo/escrow-smart-contract
Contract:     validators/escrow_milestone.ak
Blueprint:    1 validator(s), plutus v3
Typed SDK:    sdk/generated/index.ts
Tests:        1 passed, 0 failed
Deployment:    addr_test1wqhkd0vf60ud73jnyysf3j3p3y5m43ey4w6ayp526w226yxsh5zj26
Docs:         GENERATED_README.md

Why this template:
  • Selected template "Escrow with Milestone Payments", because intent category "escrow-milestone" exactly
  matches template category "escrow-milestone" and intent parsing confidence was 0.64 and match score 0.64 of
  1.00.

Why these parameters:
  • "milestoneCount" = 3, because not present (or the wrong type) in the extracted parameters; used the
  template default and the total number of milestones the escrow is split into.
```


Same Command, Different Real Contracts

```
forge build — rejected (low confidence)

$ forge build "I want to build a decentralized voting system for governance proposals"

Forge — building "I want to build a decentralized voting system for governance proposals"
(The model only classifies intent and extracts parameters — it never writes Aiken source; the deterministic
template engine does.)

No supported template matched this description with sufficient confidence (detected confidence 0.30,
required at least 0.60). Try rephrasing the description, or lower --min-confidence if this is intentional.

Supported Smart Contract Templates

1. Escrow with Milestone Payments
   Use Cases:
   • Freelancing
   • Construction
   • Project funding

2. NFT Minting with Royalties
   Use Cases:
   • NFT marketplaces
   • Creator royalties

3. Token Vesting
   Use Cases:
   • Employee token vesting
   • Investor lockups
```

Traditional Development vs. Forge

Traditional Development	Forge
Set up project manually	✓ Automated
Write boilerplate	✓ Generated
Configure the compiler	✓ Automated
Generate a typed SDK	✓ Automated
Compile the contract	✓ Automated
Run the emulator	✓ Automated
Generate deployment artifacts	✓ Automated
AI assistance	Intent parsing only — never writes contract code
Security model	Deterministic, audited templates — not AI-generated

What's Real Today vs. Roadmap

- REAL AND VERIFIED TODAY:
- Real Aiken compilation and CIP-57 blueprint parsing (not mocked)
- 3 audited, compiling contract templates (spend + mint validators)
- Confidence-gated template matching — rejects low-confidence guesses
- 150 unit tests + 8 real integration tests, all passing
- ROADMAP, HONESTLY DISCLOSED:
- ai-testgen — the eUTxO vulnerability rule engine (architected, not implemented)
- Real off-chain transaction building (ITxBuilderPort)
- Real local devnet, additional presentation layers (VS Code, web)

Why This Matters to Cardano

- eUTxO has failure modes no generic tool targets — double satisfaction has no equivalent on account-based chains
- The off-chain SDK matters more here — validators are pure functions, so SDK quality is a disproportionate share of the developer experience
- CIP-57 makes reliable codegen possible today — the machine-readable interface a typed-SDK generator needs
- Cardano hasn't had its "Hardhat moment" yet — Ethereum and Solana each inflected once scaffolding, testing, and deployment were unified

Thank you

Forge — deterministic templates, real Aiken compilation, AI kept strictly to intent parsing.

[docs/JudgePreparation.md](#) for the full FAQ · [EXECUTIVE_SUMMARY.md](#) for the one-pager